Database Management Assignment File (ENCS205)

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

Harsh Tomar (2401010219)

B. Tech CSE (Core) – 4th Semester

Section: A

Submitted To:

MS. Mansi Kajal

School of Engineering & Technology
K. R. MANGALAM UNIVERSITY
Sohna, Haryana 122103, India

Jan-2026

1.Consider following databases and draw ER diagram and convert entitiesand relationships to relation table for a given scenario:

Aim: To analyze the requirements for a College Database, design an Entity-Relationship (ER) Diagram representing Students, Sections, Subjects, and Marks, and convert the conceptual model into a Relational Database Schema.

COLLEGE DATABASE:

STUDENT (USN, SName, Address, Phone, Gender)

SEMSEC (SSID, Sem, Sec)

CLASS (USN, SSID)

SUBJECT (Subcode, Title, Sem, Credits)

IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

PROCEDURE:

Step 1: Create the Strong Entities (Rectangles) Draw four rectangles to represent the main tables in your database:

1. **STUDENT**
2. **SEMSEC** (Semester & Section)
3. **SUBJECT**
4. **IAMARKS** (Internal Assessment Marks)

Step 2: Add Attributes (Ovals) Draw ovals connected to their respective entities. Make sure to identify Primary Keys and special attributes.

- For STUDENT:
  - Draw ovals: SNAME, ADDRESS, PHONE, GENDER.
  - Draw oval USN and **underline** the text (Primary Key).
- For SEMSEC:
  - Draw ovals: SEM, SEC.
  - Draw oval SSID and **underline** the text (Primary Key).
- For SUBJECT:
  - Draw ovals: Title, SEM, CREDITS.
  - Draw oval SUBCODE and **underline** the text (Primary Key).

- For IAMARKS:

- Draw ovals: Test1, Test2, Test3. ○ Draw a Dashed/Dotted Oval for FinalIA. (This indicates it is a Derived Attribute, calculated from the other test scores).

Step 3: Create Relationships (Diamonds) Draw four diamond shapes to connect the entities.

1. Draw "CLASS" Diamond:
   - Place it between STUDENT and SEMSEC.
   - Meaning: Represents which class/section a student belongs to.
2. Draw "Secures" Diamond:
   - Place it between STUDENT and IAMARKS.
   - Meaning: Represents the marks secured by a specific student.
3. Draw "CLASS_MARKS" Diamond:
   - Place it between SEMSEC and IAMARKS.
   - Meaning: Links the marks to a specific section (to identify which class the marks belong to).
4. Draw "SCORED_IN" Diamond:
   - Place it between SUBJECT and IAMARKS.
   - Meaning: Identifies which subject the marks are for.

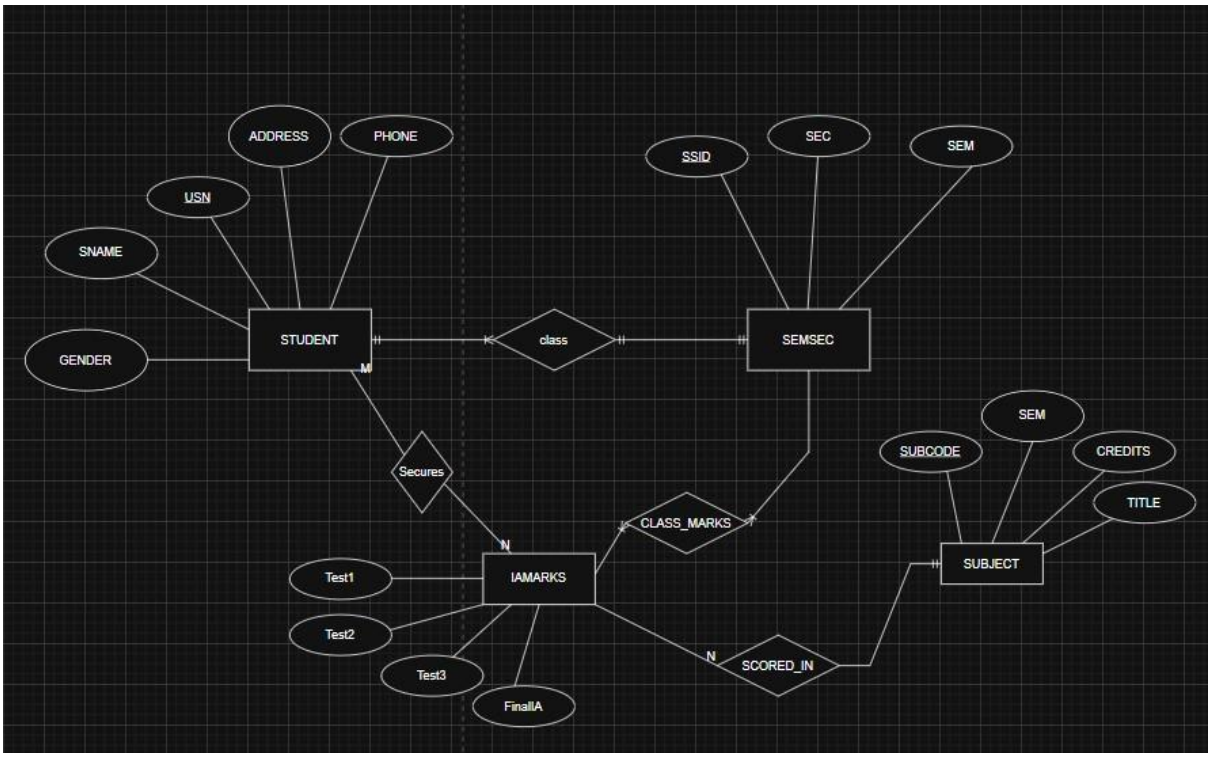
Step 4: Connect Lines & Define Cardinality (Mapping) Draw lines connecting the diamonds to the rectangles and label the cardinality (1, M, N) as shown in your image.

- For CLASS (Student - SemSec):
  - Draw a single line from STUDENT (Label 1). ○ Draw a double line (or crow's foot) from SEMSEC (Label N or (1, n)). ○ *Logic:* One Student is in One Section; One Section has Many Students.
- For Secures (Student - IAMarks): ○ Connect STUDENT to IAMARKS. ○ Label as M: N (Many-to-Many).
- For SCORED_IN (Subject - IAMarks):
  - Connect SUBJECT to IAMARKS.
  - Label as 1: N (One Subject has Many Marks entries).

- For CLASS_MARKS (SemSec - IAMarks):
  - Connect SEMSEC to IAMARKS.

- Label as 1: N (One Section has Many Marks entries).

Step 5: Final Review Check that FinalIA is dashed (derived), all Primary Keys (USN, SSID, SUBCODE) are underlined, and all 4 entities are connected through the central IAMARKS relationships



2. **Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:**

AIM: To analyze the requirements for a Company Database, design an Entity-Relationship (ER) Diagram representing Employees, Departments, Projects, and their relationships (Works On, Manages, Supervision), and map the ER diagram into a normalized Relational Database Schema.

PROCEDURE:

Step 1: Draw the 3 Main Entities (Rectangles)

Draw three rectangular boxes in the following positions:

1. DEPARTMENT: Place it in the top-middle.
2. Employee: Place it on the right side.
3. PROJECT: Place it in the bottom-left.

Step 2: Add Attributes (Ovals)

Draw ovals connected to their respective rectangles. Crucially, include the Foreign Keys as shown in your image:

- For DEPARTMENT:
  - Add ovals: DNAME, MGRSSN, MGRSTARTDATE.
  - Add oval DNO and underline it (Primary Key).
- For Employee:
  - Add ovals: NAME, ADDRESS, SALARY, SEX. ○ Add oval SSN and underline it (Primary Key).
  - Add Foreign Keys: Add ovals for SUPERSSN and DNO attached to this entity.
- For PROJECT:
  - Add ovals: PNAME, PLOCATION. ○ Add oval PNO and underline it (Primary Key).
  - Add Foreign Key: Add an oval for DNO attached to this entity.
- For Locations:
  - Draw a separate oval labeled LOOC below the Department table.

Step 3: Draw Relationships (Diamonds)

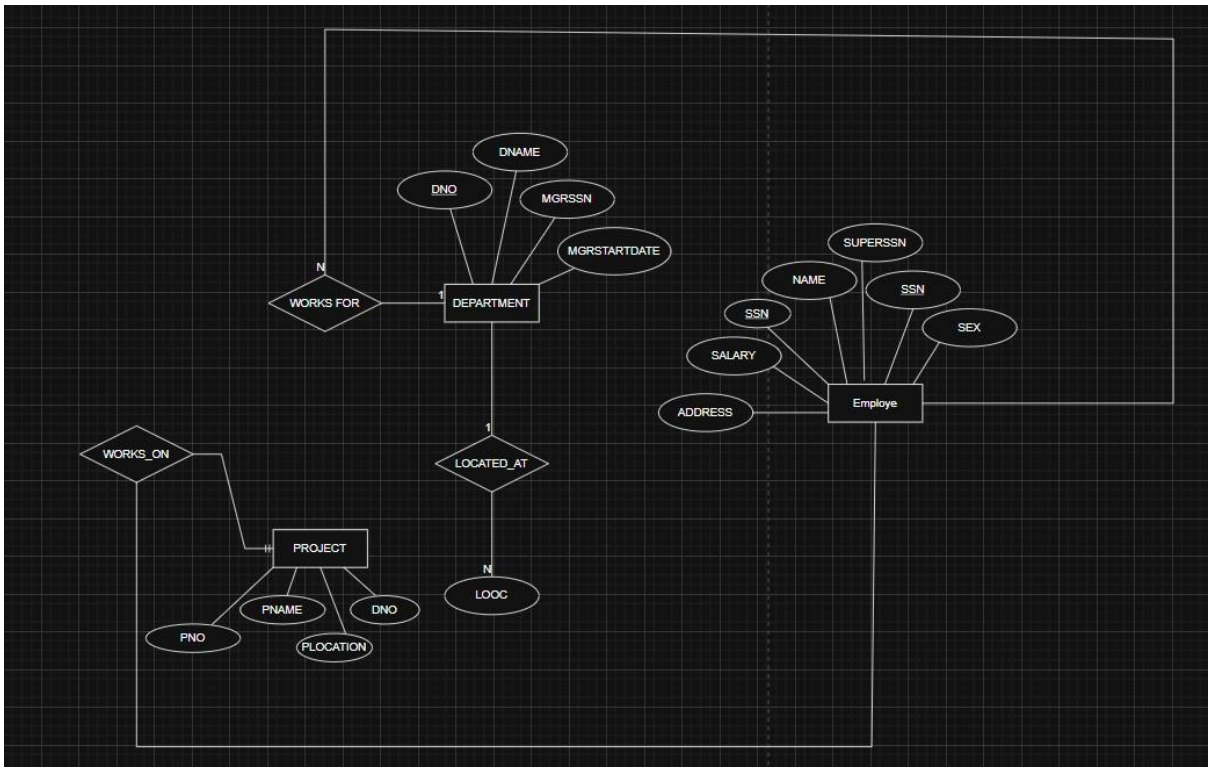
Draw three diamond shapes to represent the connections:

1. WORKS_FOR: Place this between DEPARTMENT and Employee.
2. LOCATED_AT: Place this between DEPARTMENT and the LOOC oval.
3. WORKS_ON: Place this to the left of the PROJECT table.

Step 4: Connect Lines & Define Cardinality

1. Connect WORKS_FOR:
   - Draw a line from DEPARTMENT to the diamond. Label it 1 (or N depending on your specific image logic, usually Many Employees work for 1 Dept). *In your image, N is near Employee and 1 is near Dept.*
   - Draw a line from Employee to the diamond**.**
2. Connect LOCATED_AT:
   - Draw a line from DEPARTMENT down to the diamond. Label it 1. ○ Draw a line from the diamond down to the LOOC oval. Label it N.
3. Connect WORKS_ON:
   - Draw a line from PROJECT to the diamond.

- Draw a long line from the other side of the diamond that goes Up, turns Right (across the top of the page), turns Down, and connects to the Employee table.
- This represents the Many-to-Many relationship between Employees and Projects.



3.Database Schema for a customer-sale scenario

Customer (Cust id: integer, cust_name: string)

Item (item_id: integer, item_name: string, price: integer)

Sale (bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.
2. Insert around 10 records in each of the tables.
3. List all the bills for the current date with the customer names and item numbers. 4. List the total Bill details with the quantity sold, price of the item and the final amount.
5. List the details of the customer who have bought a product which has a price>200.
6. Give a count of how many products have been bought by each customer
7. Give a list of products bought by a customer having cust_id as 5.

8. List the item details which are sold as of today.
9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.
10. Create a view which lists the daily sales date wise for the last one week

```
DROP DATABASE CUSTOMER_SALE;
CREATE DATABASE CUSTOMER_SALE;
USE CUSTOMER_SALE;
CREATE TABLE CUSTOMER(
CUST_ID INT PRIMARY KEY,
CUST_NAME VARCHAR(100)
);
CREATE TABLE ITEM (
ITEM_ID INT PRIMARY KEY,
ITEM_NAME VARCHAR(100),
PRICE INT);
CREATE TABLE SALE (
BILL_NO INT ,
BILL_DATA DATE,
CUST_ID INT,
ITEM_ID INT,
QTY_SOLD INT ,
FOREIGN KEY (CUST_ID) REFERENCES CUSTOMER(CUST_ID),
FOREIGN KEY (ITEM_ID) REFERENCES ITEM(ITEM_ID)
);
INSERT INTO
CUSTOMER(CUST_ID ,CUST_NAME)
VALUES
(1,'Aryan'),
(2,'Harsh'),
```

```
(3,'Ankit'),
(4,'Deepak'),
(5,'Hridyanshu'),
(6,'Ashok'),
(7,'Alok'),
(8,'Aman'),
(9,'Ashish'),
(10,'Mohit');
SELECT * FROM CUSTOMER;

INSERT INTO ITEM VALUES
(101,'Pen',50),
(102,'Notebook',120),
(103,'Bag',800),
(104,'Bottle',250),
(105,'Shoes',1500),
(106,'Watch',2200),
(107,'Mouse',500),
(108,'Keyboard',700),
(109,'Charger',300),
(110,'Headphones',1800);
SELECT * FROM ITEM;


 INSERT INTO SALE VALUES
(1,CURDATE(),1,103,1),
(2,CURDATE(),2,101,3),
(3,CURDATE(),2,105,2),
(4,'2026-02-15',3,104,1),
(5,'2026-02-14',4,106,1),
(6,'2026-02-13',5,102,4),
```

```
(7,CURDATE(),6,110,1),
(8,'2026-02-16',7,108,2),
(9,CURDATE(),5,109,5),
(10,'2026-02-12',8,107,1);
SELECT * FROM SALE; SELECT S.BILL_NO, C.CUST_NAME, S.ITEM_ID
FROM SALE S
JOIN CUSTOMER C ON S.CUST_ID = C.CUST_ID
WHERE S.BILL_DATA = CURDATE();

SELECT
   S.BILL_NO,
   C.CUST_NAME,
   I.ITEM_NAME,
   S.QTY_SOLD,
   I.PRICE,
   (S.QTY_SOLD * I.PRICE) AS FINAL_AMOUNT
FROM SALE S
JOIN CUSTOMER C ON S.CUST_ID = C.CUST_ID
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID;

SELECT DISTINCT C.*
FROM CUSTOMER C
JOIN SALE S ON C.CUST_ID = S.CUST_ID
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE I.PRICE > 200;

SELECT C.CUST_NAME, SUM(S.QTY_SOLD) AS TOTAL_PRODUCTS_BOUGHT
FROM CUSTOMER C
JOIN SALE S ON C.CUST_ID = S.CUST_ID
GROUP BY C.CUST_NAME;
```

```
SELECT I.ITEM_NAME, S.QTY_SOLD
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE S.CUST_ID = 5;
SELECT I.*
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
WHERE S.BILL_DATA = CURDATE();


CREATE VIEW Bill_Details AS
SELECT
   S.BILL_NO,
   S.BILL_DATA,
   S.CUST_ID,
   S.ITEM_ID,
   I.PRICE,
   S.QTY_SOLD,
   (S.QTY_SOLD * I.PRICE) AS AMOUNT
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID;

SELECT * FROM BILL_DETAILS;


CREATE VIEW WEEKLY_SALES AS
SELECT
   BILL_DATA,
   SUM(S.QTY_SOLD * I.PRICE) AS TOTAL_DAILY_SALES
FROM SALE S
JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
```

WHERE BILL_DATA >= CURDATE() - INTERVAL 7 DAY

GROUP BY BILL_DATA;

SELECT * FROM WEEKLY_SALES;

OUTPUT:

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CUST_ID	CUST_NAME
1	Aryan
2	Harsh
3	Ankit
4	Deepak
5	Hridyanshu
6	Ashok
7	Alok
8	Aman
9	Ashish
10	Mohit
NULL	NULL

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

ITEM_ID	ITEM_NAME	PRICE
101	Pen	50
102	Notebook	120
103	Bag	800
104	Bottle	250
105	Shoes	1500
106	Watch	2200
107	Mouse	500
108	Keyboard	700
109	Charger	300
110	Headphones	1800
NULL	NULL	NULL

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BILL_NO	BILL_DATA	CUST_ID	ITEM_ID	QTY_SOLD
1	2026-02-28	1	103	1
2	2026-02-28	2	101	3
3	2026-02-28	2	105	2
4	2026-02-15	3	104	1
5	2026-02-14	4	106	1
6	2026-02-13	5	102	4
7	2026-02-28	6	110	1
8	2026-02-16	7	108	2
9	2026-02-28	5	109	5
10	2026-02-12	8	107	1

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BILL_NO	CUST_NAME	ITEM_ID
1	Aryan	103
2	Harsh	101
3	Harsh	105
7	Ashok	110
9	Hridyanshu	109

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BILL_NO	CUST_NAME	ITEM_NAME	QTY_SOLD	PRICE	FINAL_AMOUNT
1	Aryan	Bag	1	800	800
2	Harsh	Pen	3	50	150
3	Harsh	Shoes	2	1500	3000
4	Ankit	Bottle	1	250	250
5	Deepak	Watch	1	2200	2200
6	Hridyanshu	Notebook	4	120	480
7	Ashok	Headphones	1	1800	1800
8	Alok	Keyboard	2	700	1400
9	Hridyanshu	Charger	5	300	1500
10	Aman	Mouse	1	500	500

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CUST_ID	CUST_NAME
1	Aryan
3	Ankit
2	Harsh
4	Deepak
8	Aman
7	Alok
5	Hridyanshu
6	Ashok

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CUST_NAME	TOTAL_PRODUCTS_BOUGHT
Aryan	1
Harsh	5
Ankit	1
Deepak	1
Hridyanshu	9
Ashok	1
Alok	2
Aman	1

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ITEM_NAME	QTY_SOLD
Notebook	4
Charger	5

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ITEM_ID	ITEM_NAME	PRICE
103	Bag	800
101	Pen	50
105	Shoes	1500
110	Headphones	1800
109	Charger	300

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BILL_NO	BILL_DATA	CUST_ID	ITEM_ID	PRICE	QTY_SOLD	AMOUNT
1	2026-02-28	1	103	800	1	800
2	2026-02-28	2	101	50	3	150
3	2026-02-28	2	105	1500	2	3000
4	2026-02-15	3	104	250	1	250
5	2026-02-14	4	106	2200	1	2200
6	2026-02-13	5	102	120	4	480
7	2026-02-28	6	110	1800	1	1800
8	2026-02-16	7	108	700	2	1400
9	2026-02-28	5	109	300	5	1500
10	2026-02-12	8	107	500	1	500

800

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

BILL_DATA	TOTAL_DAILY_SALES
2026-02-28	7250